3.2 Picasso 런타임 구성 분석

개요

Picasso 런타임은 Kusama 파라체인으로 동작하는 블록체인의 핵심 구성 요소로, Composable 런타임과 유사하지만 Kusama 생태계에 특화된 기능들과 추가적인 DeFi 팰릿들을 포함합니다. 이 문서에서는 Picasso 런타임의 Composable과의 차이점, Kusama 파라체인으로서의 특별한 설정, 그리고 런타임 업그레이드 메커니즘을 상세히 분석합니다.

3.2.1 Picasso 런타임의 Composable과의 차이점

construct_runtime! 매크로 구성 비교

Picasso 런타임은 Composable 런타임보다 더 많은 DeFi 관련 팰릿들을 포함합니다:

```
// code/parachain/runtime/picasso/src/lib.rs

construct_runtime!(

pub  enum Runtime where

Block = Block,

NodeBlock = opaque::Block,

UncheckedExtrinsic = UncheckedExtrinsic

{

// 기본 시스템 팰릿들 (Composable과 동일)

System: system = 0,

Timestamp: timestamp = 1,
```

```
Sudo: sudo = 2,

TransactionPayment: transaction_payment = 4,

AssetTxPayment : asset_tx_payment = 12,

Indices: indices = 5,

Balances: balances = 6,

// Picasso 전용 팰릿들

Identity: identity = 7, // Composable에는 없음

Multisig: multisig = 8,

// 파라체인 관련 (동일)

ParachainSystem: cumulus_pallet_parachain_system = 10,

ParachainInfo: parachain_info = 11,

// 콜레이터 지원 (동일)

Authorship: authorship = 20,

CollatorSelection: collator_selection = 21,

Session: session = 22,

Aura: aura = 23,

AuraExt: cumulus_pallet_aura_ext = 24,

// 거버넌스 - 추가 위원회들

Council: collective::<Instance1> = 30,

CouncilMembership: membership::<Instance1> = 31,

Treasury: treasury::<Instance1> = 32,

TechnicalCommittee: collective::<Instance2> = 72,

TechnicalCommitteeMembership: membership::<Instance2> = 73,
```

```
ReleaseCommittee: collective::<Instance3> = 74,

ReleaseMembership: membership::<Instance3> = 75,

// Picasso 전용 - 릴레이어 위원회

RelayerCommittee: collective::<Instance4> = 81,

RelayerCommitteeMembership: membership::<Instance4> = 82,

// DeFi 팰릿들 (Picasso 전용)

Pablo: pablo = 59, // DEX 팰릿

Oracle: oracle = 60, // 오라클 팰릿

FarmingRewards: reward::<Instance1> = 62, // 파밍 리워드

Farming: farming = 63, // 유동성 파밍

Vesting: vesting = 56, // 베스팅

// CosmWasm 지원 (Picasso 전용)

Cosmwasm: cosmwasm = 180,

// IBC 및 크로스체인 (공통이지만 설정 다름)

Ibc: pallet_ibc = 190,

Ics20Fee: pallet_ibc::ics20_fee = 191,

PalletMultihopXcmIbc: pallet_multihop_xcm_ibc = 192,

PalletXcmHelper: pallet_xcm_helper = 194,

PalletLiquidStaking: pallet_liquid_staking = 195,

// 수익 분배 (Picasso 전용)

RevenueIbc: revenue_ibc = 200,

}
```

```
);
```

주요 차이점 분석

1. DeFi 생태계 확장

Picasso는 완전한 DeFi 생태계를 제공하기 위해 추가 팰릿들을 포함합니다:

```
// Pablo DEX 구성
impl pablo::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type AssetId = CurrencyId;
type Balance = Balance;
type Convert = sp_runtime::traits::ConvertInto;
type Assets = Assets;
type LPTokenFactory = AssetsRegistry;
type PoolId = PoolId;
type PalletId = PabloPalletId;
type PoolCreationOrigin = EnsureRoot<AccountId>;
type EnableTwapOrigin = EnsureRoot<AccountId>;
type Time = Timestamp;
type TWAPInterval = TWAPInterval;
type WeightInfo = weights::pablo::WeightInfo<Runtime>;
type LPTokenExistentialDeposit = LPTokenExistentialDeposit;
}
```

```
// 파밍 리워드 시스템

impl reward::Config<FarmingRewardsInstance> for Runtime {

type RuntimeEvent = RuntimeEvent;

type SignedFixedPoint = FixedI128;

type PoolId = CurrencyId;

type StakeId = AccountId;

type CurrencyId = CurrencyId;

type MaxRewardCurrencies = ConstU32<10>;

}



// 유동성 파밍

impl farming::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type AssetId = CurrencyId;

type FarmingPalletId = FarmingPalletId;

type TreasuryAccountId = FarmingAccount;

type RewardPeriod = RewardPeriod;

type RewardPools = FarmingRewards;

type MultiCurrency = Assets;

type WeightInfo = ();

}
```

2. CosmWasm 스마트 컨트랙트 지원

Picasso는 CosmWasm 가상머신을 통해 스마트 컨트랙트 기능을 제공합니다:

```
// CosmWasm 구성 (Picasso 전용)
impl cosmwasm::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type AccountIdExtended = AccountId;
type PalletId = CosmwasmPalletId;
type MaxCodeSize = ConstU32<{ 512 * 1024 }>;
type MaxInstrumentedCodeSize = ConstU32<{ 1024 * 1024 }>;
type MaxMessageSize = ConstU32<{ 256 * 1024 }>;
type AccountToAddr = cosmwasm::AccountToAddr<Runtime>;
type AssetToDenom = cosmwasm::AssetToDenom<Runtime>;
type Balance = Balance;
type AssetId = CurrencyId;
type Assets = Assets;
type NativeAsset = NativeAssetId;
type ChainExtension = ();
type CodeUploadOrigin = EnsureRoot<AccountId>;
type InstantiateOrigin = frame_system::EnsureSigned<AccountId>;
type WeightInfo = weights::cosmwasm::WeightInfo<Runtime>;
type UnixTime = Timestamp;
type WasmVm = cosmwasm_vm_wasmi::WasmiVM;
```

```
type IbcRelayerAccount = IbcRelayerAccount;

type PalletHook = ();

type UploadWasmOrigin = EnsureRoot<AccountId>;

type ExecuteOrigin = frame_system::EnsureSigned<AccountId>;

type QueryOrigin = frame_system::EnsureSigned<AccountId>;

type MigrateOrigin = EnsureRoot<AccountId>;

type UpdateAdminOrigin = frame_system::EnsureSigned<AccountId>;

type FreezeOrigin = EnsureRoot<AccountId>;

}
```

3. 확장된 거버넌스 구조

Picasso는 릴레이어 위원회를 추가로 운영합니다:

```
// 릴레이어 위원회 구성

impl membership::Config<NativeRelayerMembership> for Runtime {

type RuntimeEvent = RuntimeEvent;

type AddOrigin = EnsureRoot<AccountId>;

type RemoveOrigin = EnsureRoot<AccountId>;

type SwapOrigin = EnsureRoot<AccountId>;

type ResetOrigin = EnsureRoot<AccountId>;

type PrimeOrigin = EnsureRoot<AccountId>;

type MembershipInitialized = RelayerCommittee;

type MembershipChanged = RelayerCommittee;

type MaxMembers = CouncilMaxMembers;
```

```
type WeightInfo = weights::membership::WeightInfo<Runtime>;

}


impl collective::Config<RelayerCollective> for Runtime {

type RuntimeOrigin = RuntimeOrigin;

type Proposal = RuntimeCall;

type RuntimeEvent = RuntimeEvent;

type MotionDuration = CouncilMotionDuration;

type MaxProposals = CouncilMaxProposals;

type MaxMembers = CouncilMaxMembers;

type DefaultVote = collective::PrimeDefaultVote;

type WeightInfo = weights::collective::WeightInfo<Runtime>;

type SetMembersOrigin = EnsureRoot<Self::AccountId>;

type MaxProposalWeight = MaxProposalWeight;

}
```

3.2.2 Kusama 파라체인으로서의 특별한 설정

네트워크 식별자 및 토큰 설정

```
// Picasso 체인 스펙에서

pub fn picasso_dev(parachain_id: ParaId) -> picasso::ChainSpec {

let mut properties = Properties::new();
```

```
properties.insert("tokenSymbol".into(), "PICA".into()); // PICA 토큰

properties.insert("tokenDecimals".into(), CurrencyId::decimals().into());

properties.insert("ss58Format".into(), picasso_runtime::SS58Prefix::get().into()

// Kusama 테스트넷 연결

Extensions {

relay_chain: "rococo_local_testnet".into(), // 개발용

para_id: u32::from(parachain_id)

}

}
```

Kusama 생태계 특화 자산 구성

```
// 제네시스 설정에서 Kusama 생태계 자산들 초기화

assets_registry: picasso_runtime::AssetsRegistryConfig {

assets: primitives::topology::Picasso::assets(), // Kusama 생태계 자산들

phantom: Default::default(),

},


// IBC 설정에서 PICA 토큰 사용

ibc: picasso_runtime::IbcConfig {

assets: vec![pallet_ibc::pallet::AssetConfig {

id: primitives::currency::CurrencyId::PICA, // 네이티브 PICA 토큰

denom: b"1".to_vec(),
```

```
}],
},
```

Pablo DEX 초기 풀 설정

```
// 제네시스에서 초기 유동성 풀 생성
let dex = picasso_runtime::PabloConfig {
pools: vec![
(root.clone(), CurrencyId(1), CurrencyId(4)), // 첫 번째 풀
(root.clone(), CurrencyId(1), CurrencyId(130)), // 두 번째 풀
],
};
```

런타임 매개변수 차이점

```
// Picasso 전용 매개변수들
parameter_types! {
// Pablo DEX 관련
pub const PabloPalletId: PalletId = PalletId(*b"pablo___");
pub const TWAPInterval: BlockNumber = 10;
pub LPTokenExistentialDeposit: Balance = NativeExistentialDeposit::get();
// 파밍 관련
pub const RewardPeriod: BlockNumber = 5; // 1분
```

```
pub  const FarmingPalletId: PalletId = PalletId(*b"mod/farm");

pub FarmingAccount: AccountId = FarmingPalletId::get().into_account_truncating()

// CosmWasm 관련

pub  const CosmwasmPalletId: PalletId = PalletId(*b"cosmwasm");

pub IbcRelayerAccount: AccountId = AccountId::from(hex!("6d6f646c70792f72656c617

}
```

3.2.3 런타임 업그레이드 메커니즘 및 거버넌스 설정

포크리스 업그레이드 시스템

Picasso는 Substrate의 포크리스 업그레이드 메커니즘을 사용합니다:

```
// System 팰릿 구성에서

impl system::Config for Runtime {

// ... 기타 설정들

type OnSetCode = cumulus_pallet_parachain_system::ParachainSetCode<Self>;

// ...

}
```

거버넌스 기반 업그레이드 프로세스

1. 제안 단계

```
// Referenda 팰릿을 통한 업그레이드 제안
```

```rust
impl pallet_referenda::Config for Runtime {
type RuntimeCall = RuntimeCall;
type RuntimeEvent = RuntimeEvent;
type Scheduler = Scheduler;
type Currency = Balances;
type SubmitOrigin = frame_system::EnsureSigned<AccountId>;
type CancelOrigin = EnsureRoot<AccountId>;
type KillOrigin = EnsureRoot<AccountId>;
type Slash = Treasury;
type Votes = Balance;
type Tally = pallet_conviction_voting::TallyOf<Runtime>;
type SubmissionDeposit = SubmissionDeposit;
type MaxQueued = ConstU32<100>;
type UndecidingTimeout = UndecidingTimeout;
type AlarmInterval = AlarmInterval;
type Tracks = TracksInfo;
type Preimages = Preimage;
type WeightInfo = pallet_referenda::weights::SubstrateWeight<Runtime>;
}
```

2. 투표 및 승인 과정

```rust
// Conviction Voting을 통한 가중 투표
```

```
impl pallet_conviction_voting::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type Currency = Balances;

type VoteLockingPeriod = VoteLockingPeriod;

type MaxVotes = ConstU32<512>;

type WeightInfo = pallet_conviction_voting::weights::SubstrateWeight<Runtime>;

type MaxTurnout = frame_support::traits::TotalIssuanceOf<Balances, Self::Account

type Polls = Referenda;

}
```

3. 스케줄링 및 실행

```
// Scheduler를 통한 업그레이드 실행 스케줄링

impl scheduler::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type RuntimeOrigin = RuntimeOrigin;

type PalletsOrigin = OriginCaller;

type RuntimeCall = RuntimeCall;

type MaximumWeight = MaximumSchedulerWeight;

type ScheduleOrigin = EnsureRoot<AccountId>;

type OriginPrivilegeCmp = EqualPrivilegeOnly;

type MaxScheduledPerBlock = MaxScheduledPerBlock;

type Preimages = Preimage;

type WeightInfo = weights::scheduler::WeightInfo<Runtime>;
```

```
}
```

마이그레이션 시스템

런타임 업그레이드 시 데이터 마이그레이션을 위한 시스템:

```
// migrations.rs에서 정의된 마이그레이션들
pub struct Migrations;

impl OnRuntimeUpgrade for Migrations {
fn on_runtime_upgrade() -> Weight {
// 버전별 마이그레이션 로직
let mut weight = Weight::zero();
// 예: 스토리지 형식 변경 마이그레이션
weight = weight.saturating_add(migrate_storage_format());
// 예: 새로운 팰릿 초기화
weight = weight.saturating_add(initialize_new_pallets());
weight
}
}
```

거버넌스 트랙 시스템

Picasso는 다양한 유형의 제안에 대해 서로 다른 거버넌스 트랙을 사용합니다:

```
// tracks.rs에서 정의된 거버넌스 트랙들

pub  struct TracksInfo;



impl pallet_referenda::TracksInfo<Balance, BlockNumber> for TracksInfo {

type Id = u16;

type RuntimeOrigin = <RuntimeOrigin as frame_support::traits::OriginTrait>::Pall

fn  tracks() -> &'static [(Self::Id, pallet_referenda::TrackInfo<Balance, BlockN

static DATA: [(u16, pallet_referenda::TrackInfo<Balance, BlockNumber>); 15] = [

// Root 트랙 - 가장 중요한 변경사항

(0, pallet_referenda::TrackInfo {

name: "root",

max_deciding: 1,

decision_deposit: 100 * UNIT,

prepare_period: 2 * HOURS,

decision_period: 28 * DAYS,

confirm_period: 24 * HOURS,

min_enactment_period: 24 * HOURS,

min_approval: pallet_referenda::Curve::LinearDecreasing {

length: Perbill::from_percent(100),

floor: Perbill::from_percent(50),

ceil: Perbill::from_percent(100),

},
```

```
min_support: pallet_referenda::Curve::LinearDecreasing {
length: Perbill::from_percent(100),
floor: Perbill::from_percent(0),
ceil: Perbill::from_percent(25),
},
}),
// 기타 트랙들...
];
&DATA[..]
}
fn  track_for(id: &Self::RuntimeOrigin) -> Result<Self::Id, ()> {
// 오리진에 따른 트랙 결정 로직
match  id {
frame_system::RawOrigin::Root => Ok(0),
// 기타 매핑들...
_ => Err(()),
}
}
}
```

3.2.4 특별한 기능들

수익 분배 시스템

Picasso는 IBC를 통한 수익 분배 시스템을 포함합니다:

```
// Revenue IBC 팰릿 구성
impl revenue_ibc::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type Balance = Balance;
type AssetId = CurrencyId;
type Assets = Assets;
type IbcTransferOrigin = EnsureRoot<AccountId>;
type WeightInfo = ();
}
```

베스팅 시스템

토큰 베스팅을 위한 전용 팰릿:

```
impl vesting::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type Currency = Balances;
type BlockNumberToBalance = ConvertInto;
type MinVestedTransfer = ExistentialDeposit;
type WeightInfo = weights::vesting::WeightInfo<Runtime>;
const MAX_VESTING_SCHEDULES: u32 = 28;
}
```

Identity 시스템

사용자 신원 확인을 위한 Identity 팰릿:

```
impl identity::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type Currency = Balances;
type BasicDeposit = BasicDeposit;
type FieldDeposit = FieldDeposit;
type SubAccountDeposit = SubAccountDeposit;
type MaxSubAccounts = MaxSubAccounts;
type MaxAdditionalFields = MaxAdditionalFields;
type MaxRegistrars = MaxRegistrars;
type Slashed = Treasury;
type ForceOrigin = EnsureRootOrHalfCouncil;
type RegistrarOrigin = EnsureRootOrHalfCouncil;
type WeightInfo = weights::identity::WeightInfo<Runtime>;
}
```

결론

Picasso 런타임은 Composable 런타임을 기반으로 하되, Kusama 생태계에 특화된 다음과 같은 특징들을 가집니다:

1. **확장된 DeFi 생태계**: Pablo DEX, 유동성 파밍, 오라클 시스템 등 완전한 DeFi 스택 제공
2. **CosmWasm 지원**: 스마트 컨트랙트 플랫폼으로서의 기능 확장

3. **고도화된 거버넌스**: 다중 위원회 시스템과 세분화된 거버넌스 트랙

4. **Kusama 생태계 통합**: PICA 토큰과 Kusama 자산들의 네이티브 지원

5. **포크리스 업그레이드**: 거버넌스 기반의 안전한 런타임 업그레이드 메커니즘

이러한 구성을 통해 Picasso는 Kusama 생태계에서 중요한 DeFi 허브 역할을 수행하며, 크로스체인 상호운용성과 혁신적인 금융 서비스를 제공합니다.